AegisFlow DevSecOps Framework

Mục tiêu của repo: nộp một bài test DevSecOps/AppSec dùng đúng một repository, một môi trường Docker Compose, một pipeline và một bộ report để chứng minh cách tích hợp security vào CI/CD local.

Trạng thái run hiện tại: target được scan là **juice-shop**, run id `pipeline-run-20260427184234`, trạng thái policy là **BLOCKED**.

5	118	50	64
Tasks Documented	Total Findings	Critical	SBOM Components

Requirement to Evidence Mapping

Yêu cầu đề bài	Bằng chứng trong repo
01 repository duy nhất	Toàn bộ Task 1–4 dùng cùng repo này, cùng pipeline shell `pipeline/run_pipeline.sh`, cùng dashboard `server.py + dashboard/` và cùng runtime `docker-compose.yml`.
`docker-compose.yml`	Khởi động service điều phối AegisFlow và ứng dụng mẫu Juice Shop để demo scan local bằng Docker.
01 file PDF duy nhất	File này được generate từ `pipeline/generate_report.py` và export bằng Playwright thành `DevSecOps_CaseStudy_Report.pdf`.
Tối thiểu 03 task	Repo hiện tại bao phủ Task 1, 2, 3, 4 và có thêm phần Task 5 trong cùng PDF.

Submission Scorecard

Task	Trạng thái	Artifact chính	Điểm nhấn kỹ thuật
Task 1	Done	`.gitlab-ci.yml`, `scan_sast.py`, `sast_results.json`	SAST được đặt sớm trong pipeline và có policy gate parse từ report.
Task 2	Done	`sca_results.json`, `iac_results.json`, `sbom.json`	Phân tách dependency risk, misconfiguration, secret exposure và SBOM.
Task 3	Done	`policy_result.json`, dashboard, `audit_log.json`	Block logic, governance view, SLA và audit trail.
Task 4	Done	Dockerfile, CI config, report section	Threat-to-mitigation mapping cho chính pipeline.
Task 5	Done	PDF section	Threat model tách khỏi target app scan theo đúng yêu cầu đề.

Cách Run Demo

1. Bật Docker Desktop.
2. Chạy `docker compose up --build` hoặc `bash run_mac.sh`.
3. Mở `http://localhost:58081`.
4. Chọn folder cần scan. Mặc định repo đang mount `./real-apps` vào `/app/workspace`.
5. Nhấn `Scan`, đợi pipeline hoàn thành, rồi xem kết quả ở dashboard và thư mục `security-results/`.

Kiến trúc và Toolchain

Dashboard UI gọi `POST /api/scan` vào `server.py`, server này chạy `pipeline/run_pipeline.sh` ở background thread, sinh JSON vào `security-results/`, sau đó đồng bộ sang `dashboard/data/` để frontend đọc. Toolchain trong artifact hiện tại là: Semgrep 1.68.0, Trivy 0.50.0, Checkov 3.2.0, Gitleaks 8.16.x, Nuclei 3.x. Với DAST, runtime scanner trong repo là **Nuclei**, còn report hợp nhất hiện đang được normalize theo model artifact **OWASP ZAP (Scan)**.

Task 1 – CI/CD Pipeline + SAST

Pipeline được thể hiện ở hai lớp: lớp local orchestrator là `pipeline/run_pipeline.sh` và lớp CI artifact là `.gitlab-ci.yml`. Các stage tối thiểu đều có mặt: **build**, **test**, **security**, **report**.

Thiết kế Stage

```
Build Validation -> Test Validation -> SAST -> SCA -> SBOM -> Secrets -> IaC -> DAST -> Report -> AI Triage -> Policy Gate
```

Vì sao chọn Semgrep cho SAST

- Cài đặt nhanh trong container Python, phù hợp bài toán local demo bằng Docker Compose.
- Hỗ trợ nhiều ngôn ngữ trong cùng repo, quan trọng vì repo chứa Node.js, Python, PHP và các sample app khác nhau.
- Xuất SARIF/JSON dễ parse để hợp nhất vào full report và policy gate.

Stage chạy SAST và lý do

SAST chạy ngay sau build/test validation. Lý do: lỗ hổng trong code nên được phát hiện sớm nhất có thể, trước khi đi tới dependency, infrastructure và runtime scanning. Với kiến trúc này, Semgrep là security gate đầu tiên trong pipeline.

Build/Test thật và Build/Test giả lập

Stage	Chế độ hiện tại	Giả lập gì	Giới hạn
Build	real	Nếu target không có build portable an toàn, framework chỉ ghi nhận structural validation thay vì build image/application đầy đủ.	Không chứng minh được ứng dụng target có build thành công như production CI runner.
Test	validation	Node.js target được validate manifest thay vì chạy full test suite, để tránh phụ thuộc dependency cache và lệnh test khác nhau giữa các app mẫu.	Không thay thế được unit/integration test thật của từng ứng dụng.

Quy tắc block và warning

Mức độ	Quyết định	Lý do
CRITICAL > 0	Block	Rủi ro có thể dẫn tới compromise trực tiếp, không phù hợp cho release pipeline.
HIGH >= 2	Block	Biểu hiện mức rủi ro hệ thống, không phải issue đơn lẻ.
MEDIUM / LOW	Warning	Không chặn ngay nhưng vẫn triage, gán SLA và ghi audit.

Ví dụ finding SAST thực tế

Loại	Tool	Severity	Finding	Location
SAST example	Semgrep (Scan)	MEDIUM	Semgrep Finding: javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection	/app/real-apps/juice-shop/data/static/codefixes/dbSchemaChallenge_1.ts:5

Task 2 – SCA + Container / IaC Security

Repo hiện tại thực hiện cả 4 nhóm scan liên quan đến Task 2: SCA bằng Trivy, IaC bằng Checkov, secrets bằng Gitleaks và SBOM bằng Syft/CycloneDX. DAST được bổ sung để kiểm chứng runtime, nhưng phần quan trọng cần chấm ở đây vẫn là dependency, secrets và IaC/container posture.

0	29	50	1
SCA	IaC	Secrets	DAST

Phân biệt loại vấn đề

Nhóm vấn đề	Ý nghĩa trong project	Fix ở đâu
Code vulnerability	Issue trong source, ví dụ SQL injection, command execution, weak CORS.	Fix trực tiếp trong file ứng dụng tại dòng bị chỉ ra bởi SAST.
Dependency vulnerability	CVE nằm ở thư viện như `body-parser`, `brace-expansion`, `express`.	Fix trong manifest/lockfile hoặc thay base image/dependency version.
Misconfiguration	Dockerfile, workflow, container privilege, healthcheck, IaC setting sai.	Fix trong Dockerfile, compose, workflow YAML, Terraform hoặc config pipeline.

Tool chọn và lý do

- Trivy: một công cụ duy nhất có thể phục vụ dependency/container use case, dễ đóng gói trong local Docker flow.
- Checkov: scan IaC và Dockerfile nhanh, output JSON dễ hợp nhất vào report.
- Gitleaks: rất phù hợp để bắt hardcoded secret trong repo mẫu dễ tổn thương.
- Syft: xuất SBOM CycloneDX chuẩn hơn so với tự dựng thủ công.

Giả lập và giới hạn

Thành phần	Chế độ	Giải thích	Giới hạn
SBOM	Hybrid	Nếu có `syft` thì dùng SBOM thật; nếu không có thì fallback sang generator CycloneDX từ `package.json`.	Simulation mode không thấy hết transitive dependency hoặc OS package.
DAST	Hybrid	Nếu có target URL thì chạy live; nếu không có target URL thì framework chuyển sang predictive mode theo thiết kế.	Predictive mode không thay thế full authenticated DAST/spidering và session-aware crawling.

Ví dụ issue thực tế từ report hiện tại

Loại	Tool	Severity	Finding	Location
Secret	Gitleaks (Scan)	CRITICAL	Hardcoded Secret: Detected a Generic API Key, potentially exposing access to various services and sensitive operations.	data/static/users.yml:88
SCA	No example available in current report.			
IaC	No example available in current report.			
DAST	OWASP ZAP (Scan)	LOW	Unknown	N/A

Task 3 – Policy Enforcement & Security Governance

Scenario bắt buộc của đề bài được đáp ứng và còn vượt ngưỡng: run hiện tại có **50 Critical**, **0 High**, **67 Medium**, **1 Low**. Vì vậy pipeline kết thúc ở trạng thái **BLOCKED** với lý do: Policy Violated: 50 CRITICAL findings detected..

Logic policy

```
If CRITICAL > 0: BLOCK
Else if HIGH >= 2: BLOCK
Else if weighted risk > threshold: WARNING
Else: PASS
```

Dashboard / Reporting View

Đối tượng xem	Thông tin chính
CISO / BOD	Pipeline status, severity distribution, số findings theo scan type, audit trail, maturity/compliance mapping.
Dev / App Team	File và line bị ảnh hưởng, CVSS, CWE/CVE, MITRE ATT&CK, remediation hint, AI triage, evidence snippet.

Quy trình xử lý và SLA

- Owner chính là team phát triển của ứng dụng được scan; security team giữ vai trò gatekeeper và reviewer.
- Critical: xử lý trong 24 giờ ở policy layer, AI triage hiện tại còn dùng deadline ngắn hơn cho dashboard.
- High: xử lý trong 72 giờ theo policy.
- Product/Engineering manager cần được thông báo khi pipeline bị block để quyết định release hold, exception hoặc hotfix window.

Exception và approval

Project có mô hình exception ở mức dữ liệu/reporting: finding giữ cờ `exception_approved`, audit log lưu actor/action, nhưng chưa có workflow UI hoàn chỉnh để enforce multi-approver production-grade. Đây là điểm phù hợp để giải thích trong case study như một giới hạn hiện tại.

Task 4 – Security of the Pipeline

Threat	Impact	Mitigation hiện có
Secrets leakage	Lộ API key/token làm mất kiểm soát scanner hoặc dịch vụ ngoài.	Repo ship `.env.example`, runtime đọc từ biến môi trường, không commit secret thật trong flow chuẩn.
Compromised CI runner	Output scan có thể bị sửa trước khi dashboard đọc.	Framework giữ lại security-results, dashboard/data và audit_log để tái dựng trace của từng run.
Malicious pipeline modification	Attacker bỏ qua stage security hoặc đổi policy gate.	Stage được khai báo rõ trong cả shell pipeline và `.gitlab-ci.yml`, dễ review diff.
Over-permissioned service account	Tăng blast radius khi runner hoặc container bị compromise.	Project chạy local, không dựa vào cloud IAM; đây vừa là điểm an toàn cho demo vừa là giới hạn so với production.
Supply chain attack	Tool hoặc dependency CI độc hại làm sai lệch kết quả scan.	Toolchain được khai báo tường minh trong Dockerfile/CI, có SBOM và SCA để nhìn thấy dependency risk.

Task 5 – Secure-by-Design & Threat Modeling

Task 5 được mô tả như một hệ thống khác với target app scan thực tế, đúng yêu cầu đề bài. Mục tiêu của phần này là chứng minh tư duy secure-by-design chứ không phải thêm một pipeline thứ hai.

```
Web App -> API -> Database -> External Payment Service
```

Threat	Design Control	CI/CD Security Check Mapping
Broken authentication / API abuse	JWT validation, RBAC, MFA cho action nhạy cảm	SAST rules cho auth bypass, API review gate và secret scan cho token handling
SQL injection vào payment records	Parameterized queries, centralized validation, allowlist schema	SAST block rule + runtime DAST validation
Secret leakage sang image hoặc repo	Secret manager, rotation policy, zero hardcoded credentials	Gitleaks + Trivy/SCA + SBOM review

Ưu điểm, nhược điểm và hướng phát triển

Nhóm	Nội dung
Ưu điểm	Một repo duy nhất, dễ chạy local; pipeline minh bạch; đa tool; dashboard có policy, audit và remediation; output JSON dễ tái sử dụng.
Nhược điểm	Build/test còn fallback nhiều; DAST artifact chưa đồng nhất tuyệt đối giữa runtime scanner và report model; server Python dùng global scan state; local mode chưa giống production runner hoàn toàn.
Mock / giả lập	Build/test fallback validation, SBOM simulation khi thiếu tool, AI triage local rule-engine khi không có API key, DAST predictive mode khi không có live target.
Hướng phát triển	Thêm authenticated DAST, real deployment mock stage, exception approval workflow, persistent datastore cho audit, signed artifact verification, runner isolation và policy-as-code tách riêng.

Kết luận

AegisFlow phù hợp để nộp bài test tuyển dụng vì người chấm có thể kiểm tra nhanh từng yêu cầu: một repo, một compose file, một pipeline, các artifact scan, logic policy, phần mock được giải thích minh bạch, và định hướng phát triển production-grade được nêu rõ thay vì che giấu giới hạn hiện tại.